Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen VI: Monografías de C#, Java, Go, Kotlin y Swift

Parte V (Bloques A–H): Máquinas virtuales gestionadas (CLR y JVM), concurrencia G-M-P en Go, Coroutines y ARC/Actores

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS) 1

1. Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ 2

1.1. Bloques A–C: Historia de C# y .NET, CLI (‘dotnet‘), CIL, tipos valor vs referencia y nulabilidad . . . 2

1.2. Bloque D: Propiedades, records, interfaces, genéricos reificados, delegados y LINQ 2

2. Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos 5

2.1. Bloques E y F: Span<T>, ref struct, ArrayPool<T>, máquinas de estados async/await, ValueTask<T> y Channels . . . 5

2.2. Bloques G y H: Ecosistema (ASP.NET Core, DI, EF Core, xUnit, BenchmarkDotNet) y los 4 proyectos en C# . . . 6

3. Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams 8

3.1. Bloques A–C: Evolución hasta Java 21 LTS, arquitectura JVM, JIT C1/C2, G1/ZGC y sintaxis fundamental . . . 8

3.2. Bloque D: Records, Sealed Classes, Pattern Matching, Genéricos (PECS), Colecciones y Streams API . . . 8

4. Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos 11

4.1. Bloques E y F: Java Memory Model, Virtual Threads (Project Loom), StructuredTaskScope y API Panama (FFM) . . . 11

4.2. Bloques G y H: Ecosistema (Maven, Gradle, JUnit 5, Spring Boot, GraalVM) y los 4 proyectos en Java . . . 12

II Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift 14

5. Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores 15

5.1. Bloques A–C: Filosofía de Go, toolchain integrada (‘go‘), tipos, valores cero, arrays vs slices y maps . . . 15

5.2. Bloque D: Métodos, composición de structs, interfaces implícitas, errores como valores y genéricos . . . 15

6. Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos 18

6.1. Bloques E y F: Planificador G-M-P, goroutines, canales, select, context.Context, sync/errgroup y análisis de escape . . . 18

6.2. Bloques G y H: Ecosistema (net/http, grpc-go, sqlc, pprof, -race) y los 4 proyectos en Go 19

7. Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform **21**
7.1. Bloques A−D: Filosofía de Kotlin, interoperabilidad JVM/KMP, null-safety, data/sealed classes y extensiones . . . 21
7.2. Bloques E−H: Funciones inline/reified, DSLs, Coroutines (suspend, CPS, Flow), ecosistema (Ktor, Compose) y proyectos . . . 21

8. Monografía de Swift 6 (Bloques A−H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores **24**
8.1. Bloques A−D: Filosofía de Swift, SIL/LLVM, SwiftPM, semántica de valor (struct/enum), Copy-on-Write y protocolos . . . 24
8.2. Bloques E−H: Gestión de memoria ARC (weak/unowned), concurrencia estricta en Swift 6 (actor, Sendable) y proyectos . . . 25

A. Notación **27**

B. Hoja de fórmulas **28**

C. Soluciones **29**
C.1. Monografía de C# (I − Bloques A−D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ . . . 29
C.2. Monografía de C# (II − Bloques E−H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos . . . 29
C.3. Monografía de Java (I − Bloques A−D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams . . . 30
C.4. Monografía de Java (II − Bloques E−H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos . . . 31
C.5. Monografía de Go (I − Bloques A−D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores . . . 32
C.6. Monografía de Go (II − Bloques E−H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos . . . 32
C.7. Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform . . . 33
C.8. Monografía de Swift 6 (Bloques A−H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores . . . 34

D. Auditoría de fuentes **35**

E. Plan de estudio **37**

Parte I

Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS)

Capítulo 1

Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

*Este capítulo inicia la monografía autónoma de **C# (.NET 8/9)** (Bloques A, B, C y D). Diseñado por Anders Hejlsberg, C# combina la ergonomía expresiva de los lenguajes funcionales modernos con un control de bajo nivel sobre la disposición de memoria (`struct`, `Span<T>`, genéricos reificados en la máquina virtual) único entre los entornos gestionados.*

Consejo

Cuando declares un `struct` en C#, márcalo como `readonly struct` (o `readonly record struct`) siempre que sea posible: si pasas un `struct` no `readonly` mediante `in` o lo almacenas en un campo `readonly`, el compilador se ve obligado a emitir copias defensivas ocultas en cada llamada a método para proteger el original.

Definición

Reificación de Genéricos en el CLR (.NET): preservación de los argumentos de tipo genérico en los metadatos CIL y en tiempo de ejecución, permitiendo que el JIT emita representaciones especializadas en memoria para cada tipo por valor (`struct`) sin encajonamiento (*boxing*).

Autoficha

¿Qué problema grave de rendimiento ocurre en una base de datos de millones de filas si un desarrollador convierte un `IQueryable<Order>` de EF Core a `IEnumerable<Order>` mediante `.AsEnumerable()` o `.ToList()` **antes** de aplicar `.Where(...)`?

1.1 Bloques A–C: Historia de C# y .NET, CLI ('dotnet'), CIL, tipos valor vs referencia y nulabilidad

En la plataforma **.NET**, el compilador Roslyn (`csc`) traduce el código fuente C# a *Common Intermediate Language* (CIL, estándar ECMA-335) almacenado en ensamblados `.dll`. Durante la ejecución, el compilador JIT **RyuJIT** traduce los métodos CIL a código máquina nativo aplicando compilación por niveles (*Tiered Compilation*) y perfilado dinámico (Dynamic PGO), o bien compila directamente a binario nativo autónomo mediante **Native AOT**. Una decisión crucial de ingeniería en C# es la distinción entre **tipos por referencia** (`class`, alojados en el montículo gestionado con una cabecera de objeto de 16 bytes en 64 bits) y **tipos por valor** (`struct`, alojados en línea en el marco de pila o directamente dentro del array/objeto contenedor sin indirección de puntero). Además, a diferencia de la JVM, el CLR **reifica los tipos genéricos**: `List<int>` o `Dictionary<long, double>` generan código máquina especializado con arrays planos de valores primitivos sin *boxing*.

1.2 Bloque D: Propiedades, records, interfaces, genéricos reificados, delegados y LINQ

C# moderno incorpora tipos de referencia anulables verificados por flujo (`<Nullable>enable</Nullable>`), tipos de datos inmutables por valor o referencia (`readonly record struct` y `record class`) con

igualdad estructural sintetizada y **LINQ** (*Language Integrated Query*). En LINQ es indispensable distinguir entre `IEnumerable<T>`, cuyos operadores reciben delegados `Func<T, bool>` evaluados en memoria en el proceso actual, e `IQueryable<T>`, cuyos operadores reciben árboles de expresión (`Expression<Func<T, bool»`) que un proveedor como Entity Framework Core inspecciona para traducirlos a consultas SQL ejecutadas en el motor de base de datos:

```
public readonly record struct SensorSample(int SensorId, double Value, long TimestampMs);

public static double ComputeHighValueAverage(ReadOnlySpan<SensorSample> samples, double
    threshold)
{
    double sum = 0.0;
    int count = 0;
    foreach (ref readonly var s in samples)
    {
        if (s.Value >= threshold) { sum += s.Value; count++; }
    }
    return count > 0 ? sum / count : 0.0;
}
```

Observa cómo `foreach (ref readonly var s in samples)` sobre un `ReadOnlySpan<SensorSample>` recorre las estructuras contiguas por referencia constante sin realizar ni una sola copia ni alojar un iterador en el montículo.

Problema resuelto

Enunciado En una aplicación financiera en C# sobre .NET de 64 bits, se mantiene un búfer de $N = 100000$ cotizaciones donde cada cotización tiene $S_{\text{val}} = 16$ B de datos (`long Timestamp`, `double Price`). Si se modela como `readonly record struct`, el array `Quote[]` ocupa 100000×16 B. Si se modelara como `class`, cada elemento añadiría 8 B de puntero en el array más 16 B de cabecera de objeto en el montículo (24 B extra por instancia). Calcula cuántos kilobytes (dividiendo entre 1000) ahorra el diseño con `struct`.

Datos

- $N = 100000$ — Número de cotizaciones en el array
- $S_{extra} = 24\,\text{B}$ — Sobrecoste por instancia de class (8 B ref + 16 B cabecera)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de memoria entre array de struct y array de class en el CLR (.NET)

$$\Delta M_{\text{kB}} = \frac{100000 \cdot 24}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 100000 instancias por 24 B y dividimos entre 1000 B/kB

$$\Delta M_{\text{kB}} = \frac{100000 \cdot 24}{1000}$$

$$\frac{(100000 \cdot 24)}{1000}$$

Resultado

$$\textit{Resultado} = 2400$$

Interpretación El diseño con readonly record struct ahorra 2400 kB (2.4 MB) y elimina 100 000 nodos del grafo del recolector de basura.

Comprobación de los pasos

- Paso 1: 2400 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) En .NET de 64 bits, la cabecera de todo objeto por referencia (`class`) consta de `ObjHeader` (8 B) y `MethodTable*` (8 B). ¿Cuántos bytes suma esta cabecera?
2 [PRB-0004] (Básico, Numérico) Un `readonly record struct Point3D(double X, double Y, double Z)` contiene tres `double` de 8 bytes. ¿Cuántos bytes ocupa cada instancia?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) Una consulta LINQ encadena `.Where(...)` y `.Take(25)` sobre una secuencia perezosa de 10 000 elementos donde se cumple el predicado en 1 de cada 4 elementos. ¿Cuántos elementos evalúa antes de detenerse (25×4)?
4 [PRB-0006] (Intermedio, Diseño) El recolector generacional del CLR divide el montículo de objetos pequeños en 3 generaciones (`Gen 0`, `Gen 1`, `Gen 2`). Si `Gen 0` tiene un presupuesto de 4 MiB y `Gen 1` de 2 MiB, ¿cuántos KiB suman las dos generaciones efímeras ($(4 + 2) \times 1024$)?

Capítulo 2

Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

Este capítulo completa la monografía de C# cubriendo los niveles Avanzado y Experto (Bloques E y F), el Ecosistema empresarial (Bloque G) y los Proyectos (Bloque H). Explicamos cómo las primitivas `Span<T>` y `ValueTask<T>` han situado a ASP.NET Core (Kestrel) entre los servidores web más rápidos del mundo.

Consejo

En todos los métodos asíncronos de entrada/salida en C#, acepta siempre un `CancellationToken ct = default` como último parámetro y pásalo hacia abajo a todas las llamadas de red y base de datos para abortar el trabajo inmediatamente cuando el cliente HTTP cierra la conexión.

Definición

Estructura restringida a pila (`ref struct` en C#): tipo por valor cuya ubicación está garantizada estáticamente por el compilador y el verificador del CLR exclusivamente en la pila de ejecución, permitiendo contener punteros interiores gestionados (`ref T` en `Span<T>`) seguros frente a compactaciones del recolector de basura.

Autoficha

¿Por qué un objeto `Span<byte>` no puede ser un campo de una clase ni persistir a través de una expresión `await` en un método `async`?

2.1 Bloques E y F: Span<T>, ref struct, ArrayPool<T>, máquinas de estados async/await, ValueTask<T> y Channels

Para permitir el acceso uniforme y sin copias a memoria gestionada (`T[]`), memoria en pila (`stackalloc`) y memoria nativa no gestionada, C# 7.2+ introdujo `Span<T>` y `ReadOnlySpan<T>`. Un `Span<T>` se declara como `ref struct`, lo que significa que el compilador y el verificador del CLR prohíben estáticamente que escape al montículo (no puede ser un campo de una `class`, ni ser encajonado como `object`, ni capturarse en una lambda, ni cruzar un punto `await`). En programación asíncrona, cada método `async` es transformado por el compilador en una estructura que implementa `IAsyncStateMachine`. Si un método `async Task<T>` completa su trabajo de forma asíncrona, aloja un objeto `Task<T>` en el montículo; pero en rutas calientes (como lecturas desde caché o sockets con datos ya presentes en el búfer del kernel) donde el 90 % de las llamadas se resuelven síncronamente, devolver `ValueTask<T>` (un `readonly struct`) evita por completo cualquier asignación en el montículo:

```
using System.Buffers;
using System.Threading;
using System.Threading.Tasks;

public sealed class CachedMetricReader
{
    private double? _cached;
    public ValueTask<double> ReadMetricAsync(CancellationToken ct)
    {
        if (_cached is double ready)
            return new ValueTask<double>(ready); // 0 bytes alojados en el montículo
        return new ValueTask<double>(FetchSlowAsync(ct));
```

```
    }
    private async Task<double> FetchSlowAsync(CancellationToken ct)
    {
        await Task.Delay(10, ct).ConfigureAwait(false);
        _cached = 42.5;
        return 42.5;
    }
}
```

Recuerda la restricción contractual de `ValueTask<T>`: nunca debe ser esperado (`await`) más de una vez ni consumido concurrentemente antes de que complete.

2.2 Bloques G y H: Ecosistema (ASP.NET Core, DI, EF Core, xUnit, BenchmarkDotNet) y los 4 proyectos en C#

En el Bloque G (Ecosistema), las aplicaciones .NET modernas se construyen sobre el contenedor de **Inyección de Dependencias (DI)** integrado en `Microsoft.Extensions.DependencyInjection`, que distingue tres ciclos de vida: `Transient` (una instancia nueva en cada resolución), `Scoped` (una instancia única por ámbito o petición HTTP, obligatoria para `DbContext` de Entity Framework Core) y `Singleton` (una única instancia segura para hilos durante toda la vida del proceso). Un antipatrón crítico (detectado automáticamente por `ValidateScopes = true` en entorno de desarrollo) es la **dependencia cautiva** (*Captive Dependency*): inyectar un servicio `Scoped` dentro de un `Singleton`, lo que retiene el servicio `Scoped` indefinidamente y provoca accesos concurrentes ilegales a `DbContext`.

Problema resuelto

Enunciado Un servicio de alta frecuencia en ASP.NET Core procesa $R = 50000$ llamadas por segundo donde el 90 % (45000 llamadas/s) se resuelven síncronamente desde memoria local y solo el 10 % (5000 llamadas/s) requieren E/S asíncrona real. Si cada objeto `Task<Result>` alojado en el montículo mide $S_{\text{task}} = 72$ B, calcula cuántos kilobytes por segundo (dividiendo entre 1000) deja de alojar en `Gen 0` el servicio al cambiar el tipo de retorno de `Task<Result>` a `ValueTask<Result>`.
Datos

- $R_{sync} = 45000\,\text{llamadas}\,\text{s}^{-1}$ — Llamadas resueltas síncronamente por segundo
- $S_{task} = 72\,\text{B}$ — Tamaño del objeto Task<Result> evitado

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de asignaciones en Gen 0 con ValueTask<T> en rutas calientes

$$\Delta A_{\text{kB/s}} = \frac{45000 \cdot 72}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 45000 llamadas síncronas por 72 B y dividimos entre 1000 B/kB

$$\Delta A_{\text{kB/s}} = \frac{45000 \cdot 72}{1000}$$

$$\frac{(45000 \cdot 72)}{1000}$$

Resultado

$$\mathit{Resultado} = 3240$$

Interpretación Se evita alojar y recolectar 3240 kB/s (3.24 MB/s) de objetos efímeros en Gen 0.
Comprobación de los pasos

- Paso 1: 3240 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) En .NET de 64 bits, un `Span<byte>` contiene una referencia interior `ref byte` (8 B) y una longitud `int` alineada a 8 B. ¿Cuántos bytes ocupa en la pila?

2 [PRB-0010] (Básico, Numérico) Si se alquila un búfer con `ArrayPool<byte>.Shared.Rent(4000)`, el pool devuelve un array cuya longitud es la siguiente potencia de dos (4096). ¿Cuántos bytes adicionales respecto a los 4000 solicitados tiene el array devuelto?

3 [PRB-0011] (Intermedio, Desarrollo por pasos) Un canal acotado `Channel.CreateBounded<Job>(new BoundedChannelOptions(128))` contiene actualmente 95 trabajos. ¿Cuántos trabajos más pueden escribirse sin suspender al productor?

4 [PRB-0012] (Intermedio, Diseño) Un contenedor de inyección de dependencias resuelve por cada petición HTTP 1 servicio `Singleton` (ya creado), 2 servicios `Scoped` y 4 servicios `Transient` (invocados 2 veces cada uno en la petición). ¿Cuántas instancias nuevas se crean durante esa petición $(2 + 4 \times 2)$?

Capítulo 3

Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

*Este capítulo inicia la monografía autónoma de **Java (Java 21 LTS)** (Bloques A, B, C y D). Java impulsa la infraestructura crítica financiera, bancaria y de comercio electrónico a escala planetaria, y con la llegada de Java 17 y Java 21 LTS ha incorporado tipos algebraicos de datos (`record` + `sealed interface`) e hilos virtuales que transforman por completo la escritura de código en la JVM.*

Consejo

Cuando operes con flujos numéricos en la API de Streams de Java, utiliza siempre las especializaciones primitivas (`IntStream`, `LongStream`, `DoubleStream` mediante `.mapToLong(...)` o `.mapToDouble(...)`) en lugar de `Stream<Integer>` o `Stream<Double>` para evitar el encajonamiento (*boxing*) de millones de objetos.

Definición

Principio PECS (Producer Extends, Consumer Super en Java): regla de varianza en el sitio de uso (*use-site variance*) según la cual una estructura parametrizada que actúa como productora de valores `T` debe tiparse de forma covariante (`? extends T`), y una que actúa como consumidora de valores `T` de forma contravariante (`? super T`).

Autoficha

¿Por qué en Java `Integer.valueOf(100) == Integer.valueOf(100)` evalúa a `true` (por la caché obligatoria de `-128` a `127` de la JLS §5.1.7), pero `Integer.valueOf(1000) == Integer.valueOf(1000)` evalúa a `false` y debe compararse siempre con `.equals()`?

3.1 Bloques A–C: Evolución hasta Java 21 LTS, arquitectura JVM, JIT C1/C2, G1/ZGC y sintaxis fundamental

Creado por James Gosling en Sun Microsystems en 1995, Java compila los ficheros `.java` mediante `javac` a código de bytes de pila (`.class`) independiente de la arquitectura física. En tiempo de ejecución, la máquina virtual **HotSpot** aplica compilación escalonada (*Tiered Compilation*): comienza interpretando el bytecode (Nivel 0), compila métodos frecuentes con el compilador rápido C1 recolectando perfiles de ejecución (Niveles $1-3$) y finalmente optimiza el código caliente con el compilador servidor **C2** (Nivel 4), capaz de desvirtualizar llamadas polimórficas monomórficas/bimórficas y aplicar análisis de escape (*Escape Analysis*) para alojar objetos efímeros en registros o escalarizarlos en la pila. Para la gestión del montículo, Java 21 ofrece **G1GC** (recolector regional por defecto) y **Generational ZGC** (`-XX:+UseZGC -XX:+ZGenerational`), un recolector concurrente con barreras de punteros coloreados (*colored pointers*) cuyas pausas *Stop-The-World* son inferiores a 1 ms incluso en montículos de varios terabytes.

3.2 Bloque D: Records, Sealed Classes, Pattern Matching, Genéricos (PECS), Colecciones y Streams API

Java 17 y Java 21 LTS introdujeron el paradigma de **Programación Orientada a Datos** (*Data-Oriented Programming*): la combinación de clases `record` (portadores transparentes e inmutables de

datos), interfaces selladas (`sealed interface ... permits ...`) y coincidencia de patrones exhaustiva en expresiones `switch` con desestructuración:

```
public sealed interface PaymentResult permits PaymentResult.Approved, PaymentResult.Declined {
    record Approved(String txId, long cents) implements PaymentResult {}
    record Declined(String reason, int retryAfterSec) implements PaymentResult {}

    static String describe(PaymentResult res) {
        return switch (res) {
            case Approved(var id, var cents) when cents >= 100_00 -> "High-value approved: " +
     id;
            case Approved(var id, var cents)                      -> "Approved: " + id + " ("
    + cents + ")";
            case Declined(var reason, var wait)                   -> "Declined (" + reason + "
    ), wait " + wait + "s";
        };
    }
}
```

En el sistema de tipos genéricos de Java (implementado con borrado de tipos a `Object` o a su cota superior para mantener compatibilidad binaria), la varianza en sitios de uso se rige por la regla **PECS** (*Producer Extends, Consumer Super*, formulada por Joshua Bloch): usa `List<? extends T>` cuando solo leas elementos `T` de la estructura productora, y `Comparator<? super T>` o `Collection<? super T>` cuando insertes o consumas elementos `T`.

Problema resuelto

Enunciado En la JVM HotSpot de 64 bits con punteros comprimidos activos (`-XX:+UseCompressedOops`), un objeto `Double` encajonado (*boxed*) ocupa 24 B (12 B de cabecera + 8 B del `double` + 4 B de relleno a múltiplo de 8), más 4 B de la referencia en un `ArrayList<Double>` (28 B por número), mientras que un array primitivo `double[]` o un `DoubleStream` ocupa únicamente 8 B por número. Calcula cuántos megabytes (dividiendo entre 10^6) se ahorran al almacenar $N = 2000000$ de valores en `double[]` frente a `ArrayList<Double>`.
Datos

- $N = 2000000$ — Dos millones de números en coma flotante
- $S_{boxed} = 28\,\text{B}$ — 24 B del objeto Double + 4 B de referencia comprimida
- $S_{prim} = 8\,\text{B}$ — 8 B por elemento en double[]

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Sobrecoste de memoria por boxing en la JVM frente a tipos primitivos

$$\Delta M_{\text{MB}} = \frac{2 \cdot 10^6 \cdot (28 - 8)}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 2 millones de elementos por los 20 B ahorrados y dividimos entre 10^6

$$\Delta M_{\text{MB}} = \frac{2 \cdot 10^6 \cdot (28 - 8)}{10^6}$$

$$\frac{(2000000 \cdot (28 - 8))}{1000000}$$

Resultado

$$\textit{Resultado} = 40$$

Interpretación Usar arrays o flujos primitivos ahorra exactamente 40 MB de montículo y elimina 2 millones de objetos del recolector de basura.
Comprobación de los pasos

- Paso 1: 40 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En la JVM HotSpot con Compressed OOPs, la cabecera de un objeto ordinario mide 8 B (`Mark Word`) más 4 B (`Klass Pointer`). ¿Cuántos bytes mide la cabecera antes de alinear campos?
2 [PRB-0016] (Básico, Desarrollo por pasos) En `HashMap` de Java 8+, cuando un cubo (*bucket*) alcanza el umbral `TREEIFY_THRESHOLD = 8` colisiones (con capacidad de tabla ≥ 64), la lista enlazada se transforma en un árbol rojo-negro. Si hay 256 elementos colisionando en ese mismo cubo, ¿cuál es la altura máxima $2\log_2(256)$ a comparar frente a los 256 saltos lineales?
3 [PRB-0017] (Intermedio, Desarrollo por pasos) Un `ConcurrentHashMap` redimensiona su tabla de 1024 a 2048 cubos manteniendo el factor de carga en 0,75. ¿Cuántas entradas admite la nueva tabla antes del siguiente redimensionado ($2048 \times 3/4$)?
4 [PRB-0018] (Intermedio, Diseño) Si el tamaño de un montículo de la JVM supera los 32 GiB y desactiva Compressed OOPs, cada referencia pasa de 4 B a 8 B. En un grafo en memoria con 50 millones de referencias, ¿cuántos megabytes (10^6 B) adicionales consumen las referencias?

Capítulo 4

Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

*Este capítulo completa la monografía de Java (Bloques E, F, G y H). Analizamos con rigor la innovación más importante de la JVM en la última década: los **Hilos Virtuales** (*Virtual Threads*, JEP 444 en Java 21 LTS), que reconcilian el modelo sencillo de un hilo por petición (*thread-per-request*) con la escalabilidad masiva de la E/S no bloqueante.*

Consejo

No reutilices en un pool (`ThreadPool`) los hilos virtuales: los hilos virtuales son baratos y efímeros; crea uno nuevo por cada tarea con `Executors.newVirtualThreadPerTaskExecutor()` y usa un `Semaphore` cuando necesites limitar el número de llamadas simultáneas a una base de datos externa.

Definición

Hilo Virtual en la JVM (Virtual Thread, JEP 444): instancia de `java.lang.Thread` planificada en espacio de usuario por la máquina virtual sobre un conjunto reducido de hilos portadores del sistema operativo (*carrier threads*), cuya pila de marcos crece y se encoge dinámicamente en el montículo durante bloqueos de E/S.

Autoficha

¿Qué es el *anclaje de hilo portador* (*carrier thread pinning*) cuando un hilo virtual se bloquea en E/S dentro de un bloque `synchronized` tradicional en Java 21 y por qué sustituirlo por `ReentrantLock` evita el anclaje?

4.1 Bloques E y F: Java Memory Model, Virtual Threads (Project Loom), StructuredTaskScope y API Panama (FFM)

El **Java Memory Model (JMM)** define formalmente cuándo las escrituras de un hilo son visibles para otro hilo a través de la relación transitiva *happens-before*: la liberación de un monitor (`synchronized` o `Lock.unlock()`) *happens-before* toda adquisición posterior del mismo monitor, y una escritura en un campo `volatile` *happens-before* toda lectura posterior de ese campo. En Java 21 LTS, **Project Loom** introdujo los **Virtual Threads** (`Thread.ofVirtual().start(...)` o `Executors.newVirtualThreadPerTaskExecutor()`): hilos gestionados por la JVM que se montan sobre un pequeño grupo de hilos del sistema operativo (*carrier threads* en un `ForkJoinPool`). Cuando el código ejecutado dentro de un hilo virtual realiza una operación bloqueante de red o base de datos (como `Socket.read()` o `Thread.sleep()`), el runtime de la JVM **desmonta** automáticamente la pila de continuación del hilo virtual hacia el montículo y deja libre al hilo portador del sistema operativo para ejecutar otro hilo virtual:

```
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public final class VirtualThreadDispatcher {
    public static long runConcurrentTasks(int taskCount) throws Exception {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
```

```
            var futures = IntStream.range(0, taskCount)
                .mapToObj(i -> executor.submit(() -> (long) i * 2L))
                .toList();
            long total = 0L;
            for (var f : futures) { total += f.get(); }
            return total;
        }
    }
}
```

Por la Ley de Little ($L = \lambda \cdot W$), un servidor ya no está limitado por el número de hilos de plataforma ($\approx$ 200–1000), sino que puede sostener cientos de miles de peticiones bloqueadas en E/S escribiendo código imperativo directo, fácil de depurar y con trazas de pila legibles.

4.2 Bloques G y H: Ecosistema (Maven, Gradle, JUnit 5, Spring Boot, GraalVM) y los 4 proyectos en Java

Asimismo, Java 21/22 reemplaza a JNI y `sun.misc.Unsafe` con la **Foreign Function & Memory API (Project Panama)**: `Arena.ofConfined()` y `MemorySegment` permiten gestionar memoria nativa fuera del montículo (*off-heap*) con límites espaciales y temporales comprobados determinísticamente. En el Bloque G (Ecosistema), estudiamos la gestión declarativa de proyectos con `Maven` (`pom.xml`) y `Gradle` (`build.gradle.kts`), pruebas unitarias y de integración con `JUnit 5` y `Testcontainers`, microservicios con `Spring Boot 3` (`spring.threads.virtual.enabled=true`) y compilación nativa con `GraalVM Native Image`.

Problema resuelto

Enunciado Según la Ley de Little ($\lambda_{\text{máx}} = L_{\text{máx}}/W$), un microservicio donde cada petición pasa $W = 0{,}25$ s esperando respuestas de APIs externas está limitado con hilos tradicionales de plataforma a $L_{\text{plat}} = 400$ hilos (caudal máximo $400/0{,}25 = 1600$ req/s). Al migrar a **Virtual Threads** en Java 21 soportando $L_{\text{virt}} = 20000$ hilos virtuales simultáneos con la misma latencia $W = 0{,}25$ s, calcula cuántas peticiones por segundo adicionales puede sostener el servidor ($20000 / 0.25 - 400 / 0.25$).
Datos

- $L_{plat} = 400\,\text{hilos}$ — Pool clásico de hilos del sistema operativo
- $L_{virt} = 20000\,\text{hilos}$ — Hilos virtuales simultáneos en Java 21
- $W = 0{,}25\,\text{s}$ — Tiempo medio de espera de E/S por petición

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Incremento de caudal máximo por Ley de Little con Virtual Threads (JEP 444)

$$\Delta\lambda = \frac{20000}{0{,}25} - \frac{400}{0{,}25}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el caudal antiguo (1600 req/s) al nuevo caudal (80000 req/s)

$$\Delta\lambda = \frac{20000}{0{,}25} - \frac{400}{0{,}25}$$

$$\left(\frac{20000}{0{,}25} - \frac{400}{0{,}25}\right)$$

Resultado

$$Resultado = 78400$$

Interpretación El servidor incrementa su capacidad en 78 400 peticiones/s (pasando de 1600 req/s a 80 000 req/s).

Comprobación de los pasos

- Paso 1: 78400 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) Por la Ley de Little ($L = \lambda \cdot W$), si un sistema recibe $\lambda = 2500$ peticiones/s y cada una tarda $W = 0{,}2$ segundos, ¿cuántas peticiones hay en vuelo simultáneamente en promedio?

2 [PRB-0022] (Básico, Numérico) En una CPU de 16 núcleos lógicos, el `ForkJoinPool` portador de hilos virtuales crea por defecto un hilo portador (*carrier thread*) por núcleo. ¿Cuántos hilos del sistema operativo mantienen activos los hilos virtuales?

3 [PRB-0023] (Intermedio, Desarrollo por pasos) En el código `VirtualThreadDispatcher.runConcurrentTasks(100)`, se suma `i * 2` para $i = 0, 1, \ldots, 99$. ¿Cuál es el valor exacto de la suma (99×100)?

4 [PRB-0024] (Intermedio, Diseño) Una `Arena.ofConfined()` de Project Panama reserva 64 segmentos nativos de 4096 bytes cada uno. ¿Cuántos bytes de memoria *off-heap* se liberan deterministicamente al cerrarse el bloque `try-with-resources`?

Parte II

Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift

Capítulo 5

Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

*Este capítulo inicia la monografía autónoma de **Go (Golang 1.22+)** (Bloques A, B, C y D). Go es el lenguaje fundacional de la infraestructura nativa en la nube (Docker, Kubernetes, etcd, Prometheus, Terraform, Caddy). Estudiaremos por qué su deliberada economía de conceptos produce sistemas distribuidos mantenibles por grandes equipos.*

Consejo

Sigue el axioma idiomático de Go: *«Acepta interfaces (pequeñas, como `io.Reader`), devuelve estructuras concretas»*, y evalúa siempre los argumentos de `defer` teniendo en cuenta que se capturan en el momento en que se ejecuta la sentencia `defer`, no cuando retorna la función.

Definición

Expresión de rebanada completa (`s[i:j:k]` en Go): operación de sub-slicing que produce una nueva rebanada con longitud `j - i` y capacidad restringida a `k - i`, forzando que cualquier `append` posterior sobre la sub-rebanada aloje un nuevo array en lugar de sobrescribir los elementos a partir del índice `k`.

Autoficha

¿Por qué una función en Go que declara `var p *MyCustomError = nil; return p` como valor de retorno de tipo `error` hace que el llamador observe `err != nil`?

5.1 Bloques A–C: Filosofía de Go, toolchain integrada (‘go‘), tipos, valores cero, arrays vs slices y maps

Diseñado en Google en 2007 por Robert Griesemer, Rob Pike y Ken Thompson, Go nació como respuesta a los tiempos de compilación masivos y la complejidad accidental de C++ y Java en servidores multinúcleo. La herramienta unificada `go` integra compilación, gestión de módulos (`go.mod` / `go.sum` con selección de versión mínima, *Minimal Version Selection*), formateo canónico (`gofmt`), análisis estático (`go vet`), pruebas y *fuzzing* nativo (`go test -fuzz`). En Go, toda variable no inicializada explícitamente recibe su **valor cero** determinista (`0`, `false`, `""`, `nil`), y los principios de diseño recomiendan que el valor cero de una estructura sea inmediatamente utilizable sin constructor (como `var mu sync.Mutex` o `var buf bytes.Buffer`). Un `slice []T` es un descriptor de 24 bytes (en 64 bits) formado por un puntero al array subyacente, la longitud `len` y la capacidad `cap`. Si dos rebanadas comparten el mismo array subyacente y una ejecuta `append` sin superar `cap`, modifica el array compartido; para independizar la capacidad al hacer sub-slicing se utiliza la expresión de tres índices `s[low:high:max]`.

5.2 Bloque D: Métodos, composición de structs, interfaces implícitas, errores como valores y genéricos

Go prescinde por completo de la herencia de clases y la sustituye por **composición de estructuras** y **interfaces satisfechas implícitamente**: un tipo satisface una interfaz como `io.Reader` (`Read(p []byte) (n int, err error)`) simplemente implementando sus métodos, sin escribir nunca

`implements`. Esto permite definir interfaces pequeñas de uno o dos métodos en el paquete consumidor en lugar de acoplarse al paquete proveedor. Asimismo, Go trata los errores como valores ordinarios devueltos en la última posición de retorno `(val, error)`, encadenados con `fmt.Errorf("contexto: %w", err)` e inspeccionados mediante `errors.Is` y `errors.As`:

```
package storage

import (
    "errors"
    "fmt"
    "io"
)

var ErrEmptyPayload = errors.New("empty payload")

func ReadExactHeader(r io.Reader, size int) ([]byte, error) {
    if size <= 0 {
        return nil, ErrEmptyPayload
    }
    buf := make([]byte, size)
    if _, err := io.ReadFull(r, buf); err != nil {
        return nil, fmt.Errorf("read exact header (%d bytes): %w", size, err)
    }
    return buf, nil
}
```

Un matiz técnico esencial de las interfaces en Go es su representación interna de dos palabras (`tab` / tipo dinámico y `data` / valor dinámico): una interfaz solo es `== nil` si **ambos** punteros internos son nulos; asignar un puntero tipado `(*MyError)(nil)` a una variable `var err error` produce una interfaz **no nula** (`err != nil`), por lo que las funciones deben devolver siempre el literal `nil` directamente cuando no hay error.

Problema resuelto

Enunciado En un programa en Go, se crea un slice `base := make([]int64, 100, 200)` (cada `int64` ocupa 8 B) y se obtiene la sub-rebanada de tres índices `sub := base[20:60:80]`. Calcula la suma de la longitud `len(sub)` ($60 - 20$) más la capacidad `cap(sub)` ($80 - 20$) multiplicada por los 8 B de cada elemento.

Datos

- $i = 20$ — Índice inicial
- $j = 60$ — Índice final de longitud
- $k = 80$ — Índice límite de capacidad

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Cálculo de len y cap en expresiones de rebanada de tres índices s[i:j:k] en Go

$$R = ((60 - 20) + (80 - 20)) \cdot 8$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos len (40) + cap (60) = 100 y multiplicamos por 8 bytes

$$R = ((60 - 20) + (80 - 20)) \cdot 8$$

$$(((60 - 20) + (80 - 20)) \cdot 8)$$

Resultado

$$Resultado = 800$$

Interpretación La longitud de sub es 40, su capacidad es 60 y la métrica ponderada por 8 bytes es 800.

Comprobación de los pasos

- Paso 1: 800 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En Go de 64 bits, el encabezado de un slice (`SliceHeader`) consta de `Data` (8 B), `Len` (8 B) y `Cap` (8 B). ¿Cuántos bytes ocupa en la pila?

2 [PRB-0028] (Básico, Numérico) En Go, cada cubo (*bucket* `bmap`) de un `map[K]V` almacena exactamente 8 pares clave-valor. Si un mapa tiene $2^6 = 64$ cubos, ¿cuántos pares clave-valor caben en esos 64 cubos sin cubos de desbordamiento (64×8)?

3 [PRB-0029] (Intermedio, Desarrollo por pasos) Para la rebanada `s[15:45:50]`, ¿cuántos elementos pueden añadirse con `append` antes de superar la capacidad y forzar la reserva de un nuevo array ($(50 - 15) - (45 - 15)$)?

4 [PRB-0030] (Intermedio, Diseño) Una interfaz `io.Reader` en Go se representa como un par de palabras `(itab*, data*)` de 8 bytes cada una. ¿Cuántos bytes ocupan 500 descriptores de interfaz en un slice `[]io.Reader`?

Capítulo 6

Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

*Este capítulo completa la monografía de Go (Bloques E, F, G y H). Analizamos el motor de ejecución de Go: su planificador de **goroutines G-M-P**, los patrones de canales y `context.Context` para evitar fugas de goroutines, el recolector de basura tricolor concurrente de baja latencia y las herramientas integradas de diagnóstico de producción (`pprof`, `trace`, `-race`).*

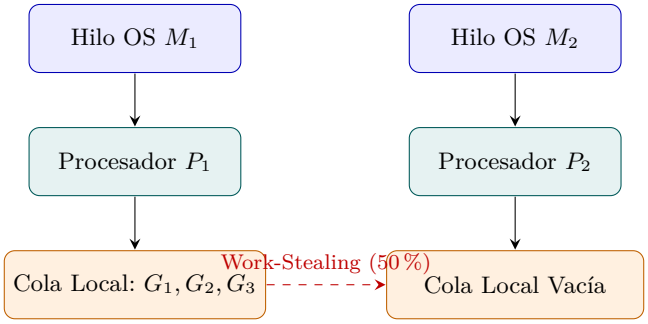


Figura 1. Arquitectura del planificador M:N de Go: cada contexto lógico P (GOMAXPROCS) posee una cola local sin cerrojos de Goroutines (G) y se vincula a un hilo del sistema operativo (M) para ejecutar código Go o robar la mitad de la cola de otro P.

Consejo

Antes de lanzar cualquier `go func() { ... }()`, responde siempre a dos preguntas: (1) ¿bajo qué condición exacta terminará esta goroutine (por ejemplo, cancelación de `ctx.Done()` o cierre del canal de entrada)? y (2) ¿quién esperará a que termine (`sync.WaitGroup` o `errgroup.Group`) antes de cerrar el proceso?

Definición

Fuga de Goroutines (Goroutine Leak): defecto de recursos en Go en el que una goroutine queda suspendida permanentemente en una operación de envío o recepción de canal que ningún otro participante completará jamás, impidiendo que el recolector de basura libere su pila y las variables referenciadas por ella.

Autoficha

¿Qué ocurre exactamente en tiempo de ejecución en Go cuando una goroutine intenta enviar un valor a un canal cerrado (`close(ch)`) frente a cuando intenta recibir de un canal cerrado?

6.1 Bloques E y F: Planificador G-M-P, goroutines, canales, select, context.Context, sync/errgroup y análisis de escape

Una **goroutine** (`go fn()`) es un hilo de ejecución ligero gestionado por el runtime de Go cuya pila inicial mide tan solo **2 KiB** (2048 B) y crece o decrece dinámicamente según la profundidad de llamadas. Como muestra la Figura \ref{fig:fig-go-gmp-scheduler}, el planificador M:N multiplexa N goroutines (`G`) sobre M hilos del sistema operativo (`M`) utilizando hasta `GOMAXPROCS` procesadores lógicos (`P`). Cada `P` mantiene una cola local de hasta 256 goroutines ejecutables; cuando la cola de un `P` se vacía, intenta

robar la mitad de las goroutines de la cola de otro `P` elegido al azar (*work-stealing*) o toma trabajo de la cola global. La comunicación entre goroutines sigue el cálculo **CSP** (*Communicating Sequential Processes*): *«No compartas memoria para comunicarte; comunícate para compartir memoria»*. Para evitar que una goroutine quede bloqueada eternamente intentando enviar en un canal cuyo receptor ya ha abandonado la operación (*goroutine leak*), todo ciclo concurrente debe seleccionar también sobre `<-ctx.Done()` de un `context.Context`:

```
package worker

import "context"

func FanOutGenerator(ctx context.Context, items []int, out chan<- int) error {
    for _, v := range items {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case out <- v * 2:
        }
    }
    return nil
}
```

A partir de Go 1.22, las variables de bucle `for i, v := range items` tienen ámbito por iteración en lugar de por bucle completo, eliminando el error histórico de capturar la misma variable `v` en múltiples clausuras concurrentes.

6.2 Bloques G y H: Ecosistema (net/http, grpc-go, sqlc, pprof, -race) y los 4 proyectos en Go

En el Nivel Experto (Bloque F), el compilador de Go decide automáticamente si una variable local se aloja en la pila de 2 KiB de la goroutine o en el montículo gestionado mediante **análisis de escape** (*escape analysis*), inspeccionable con `go build -gcflags="-m"`. En el Bloque G y Bloque H, la biblioteca estándar de Go es tan completa que permite construir servidores HTTP/2 de producción con enrutamiento por patrones (`net/http` en Go 1.22+), observabilidad (`log/slog`, `runtime/pprof`), clientes SQL con pools de conexiones (`database/sql`) y pruebas de concurrencia instrumentadas con ThreadSanitizer (`go test -race`). Los cuatro proyectos del bloque incluyen un proxy inverso con balanceo *round-robin* y *health checks*, un grupo de trabajadores (*worker pool*) con apagado ordenado (`os/signal` + `context.WithTimeout`), un limitador de tasa distribuido y un servidor gRPC.

Problema resuelto

Enunciado Cada goroutine recién creada en Go 1.22 reserva una pila inicial de $S_G = 2$ KiB (2048 B), mientras que un hilo nativo de sistema operativo reserva por defecto una pila de $S_M = 2048$ KiB (2 MiB). Si un servidor de telemetría en Go mantiene $N = 100000$ goroutines activas sobre $M = 16$ hilos del sistema operativo, calcula cuántos mebibytes totales (dividiendo los KiB entre 1024) ocupan las pilas de las 100 000 goroutines más los 16 hilos OS.

Datos

- $N = 100000$ — Número de goroutines concurrentes
- $S_G = 2\,\text{KiB}$ — Pila inicial por goroutine
- $M = 16$ — Hilos del sistema operativo de 2048 KiB

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Consumo de memoria de pilas en el modelo M:N de Go

$$M_{\text{MiB}} = \frac{100000 \cdot 2 + 16 \cdot 2048}{1024}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Sumamos los KiB de las goroutines (200000 KiB) y de los 16 hilos OS (32768 KiB) y dividimos entre 1024

$$M_{\text{MiB}} = \frac{100000 \cdot 2 + 16 \cdot 2048}{1024}$$

$$\frac{((100000 \cdot 2) + (16 \cdot 2048))}{1024}$$

Resultado

$$Resultado = 227{,}3125$$

Interpretación Las 100 000 goroutines más los 16 hilos del sistema operativo consumen tan solo 227.3125 MiB de memoria de pila.
Comprobación de los pasos

- Paso 1: 227,312 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0033] (Básico, Numérico) ¿Cuántos bytes mide la pila inicial de una goroutine en Go (2×1024)?
2 [PRB-0034] (Básico, Numérico) En el planificador de Go, un procesador P_1 tiene 180 goroutines en su cola local y un procesador inactivo P_2 le roba la mitad. ¿Cuántas goroutines roba P_2?
3 [PRB-0035] (Intermedio, Desarrollo por pasos) Un canal con búfer `ch := make(chan uint64, 512)` almacena elementos de 8 bytes. Si hay 300 elementos encolados en el búfer, ¿cuántos elementos adicionales caben antes de bloquear al emisor?
4 [PRB-0036] (Intermedio, Diseño) En el problema anterior, ¿cuántos bytes ocupa el búfer circular completo de 512 elementos `uint64` (512×8)?

Capítulo 7

Monografía de Kotlin (Bloques A–H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

*Este capítulo ofrece la monografía completa (Bloques A – H) de **Kotlin (2.0+)**. Desarrollado por JetBrains, Kotlin es el lenguaje oficial prioritario para el desarrollo en Android, una potencia en microservicios de servidor sobre la JVM (Spring Boot, Ktor) y un lenguaje multiplataforma nativo (**Kotlin Multiplatform**, KMP).*

Consejo

Cuando captures excepciones genéricas (`catch (e: Throwable)` o `catch (e: Exception)`) dentro de una corrutina de Kotlin, **relanza siempre `CancellationException`** (`if (e is CancellationException) throw e`): tragarse `CancellationException` rompe el mecanismo de cancelación cooperativa de la concurrencia estructurada.

Definición

Transformación CPS de funciones `suspend` en Kotlin: reescritura realizada por el compilador en la que toda función `suspend fun f(a: A): R` se compila en bytecode a `Object f(A a, Continuation<? super R> completion)`, donde `completion` almacena el índice `label` del estado actual y las variables locales vivas entre puntos de suspensión.

Autoficha

¿Qué diferencia de comportamiento ante el fallo de una corrutina hija existe entre `coroutineScope { ... }` y `supervisorScope { ... }` en Kotlin Coroutines?

7.1 Bloques A–D: Filosofía de Kotlin, interoperabilidad JVM/KMP, null-safety, data/sealed classes y extensiones

Kotlin resuelve las limitaciones históricas de Java manteniendo una interoperabilidad del 100 % a nivel de bytecode. Su sistema de tipos incorpora nulabilidad en la raíz de la jerarquía: `String` no puede almacenar `null`, mientras que `String?` exige comprobación explícita mediante el operador de llamada segura `?.`, el operador Elvis `?:` o *smart casts* tras verificar `if (x != null)`. Para modelar dominios sin sobrecoste de ejecución, `@JvmInline value class UserId(val raw: Long)` envuelve un tipo primitivo en compilación pero se emite como un `long` desnudo en el bytecode de la JVM. Asimismo, dado que en Kotlin las funciones de orden superior se usan intensivamente, la palabra clave `inline` inserta el cuerpo de la función y el de las lambdas recibidas directamente en el sitio de llamada, eliminando la creación de objetos clausura y permitiendo preservar el tipo genérico en ejecución con `reified T`.

7.2 Bloques E–H: Funciones inline/reified, DSLs, Coroutines (suspend, CPS, Flow), ecosistema (Ktor, Compose) y proyectos

La concurrencia en Kotlin se fundamenta en **Coroutines** estructuradas a partir de una única palabra clave en el lenguaje: `suspend`. El compilador K2 transforma cada `suspend fun fetch(id: Long):`

`Profile` mediante **Continuation-Passing Style (CPS)** en una función que recibe un parámetro adicional oculto `Continuation<Profile>` implementado como una máquina de estados sin alojar un hilo por espera:

```
import kotlinx.coroutines.*

data class NodeHealth(val nodeId: Int, val latencyMs: Long)

suspend fun aggregateClusterLatency(nodeIds: List<Int>): Long = coroutineScope {
    val deferreds: List<Deferred<Long>> = nodeIds.map { id ->
        async(Dispatchers.Default) {
            // Simulación de consulta paralela por nodo:
            id.toLong() * 5L
        }
    }
    deferreds.awaitAll().sum()
}
```

Dentro de `coroutineScope { ... }`, la concurrencia es estrictamente jerárquica: la función no retorna hasta que todas las corrutinas hijas lanzadas con `launch` o `async` han terminado, y si una sola falla con una excepción, todas las demás hermanas reciben cancelación cooperativa inmediata. Para secuencias asíncronas de múltiples valores, Kotlin distingue entre `Flow<T>` (flujo frío perezoso con contrapresión natural por suspensión) y `StateFlow<T>` / `SharedFlow<T>` (flujos calientes observables en arquitecturas reactivas como Jetpack Compose).

Problema resuelto

Enunciado En una aplicación en Kotlin sobre la JVM, una función de orden superior que captura una variable local del entorno se invoca en un bucle caliente $N = 500000$ veces por segundo. Si la función NO está marcada como `inline`, cada invocación aloja una instancia de clase anónima de $S_{\text{lambda}} = 24$ B. Al marcar la función de orden superior con `inline`, el alojamiento cae a 0 B. Calcula cuántos megabytes por segundo (dividiendo entre 10^6) deja de alojar el bucle gracias a `inline`.
Datos

- $N = 500000$ — Invocaciones por segundo
- $S_\lambda = 24\,\text{B}$ — Tamaño del objeto clausura capturado

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de montículo mediante expansión inline de lambdas con captura en Kotlin

$$\Delta M_{\text{MB/s}} = \frac{500000 \cdot 24}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las 500000 invocaciones por 24 B y dividimos entre 10^6

$$\Delta M_{\text{MB/s}} = \frac{500000 \cdot 24}{10^6}$$

$$\frac{(500000 \cdot 24)}{1000000}$$

Resultado

$$\textit{Resultado} = 12$$

Interpretación La palabra clave inline ahorra exactamente 12 MB/s de asignaciones efímeras en el montículo.
Comprobación de los pasos

- Paso 1: 12 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** En la función `aggregateClusterLatency` con `nodeIds = [1, 2, 3, 4, 5]` donde cada nodo devuelve `id * 5`, ¿cuál es la suma total devuelta $((1+2+3+4+5)\times 5)$?
2 [**PRB-0040**] **(Básico, Numérico)** Un `@JvmInline value class Meter(val v: Double)` se almacena en un array primitivo de 1000 elementos. ¿Cuántos bytes ocupan los 1000 elementos `double` (1000×8)?
3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** Un `Flow<Int>` emite 60 elementos a razón de 1 elemento cada 10 ms y el colector aplica `.buffer(10)` tardando 25 ms por elemento. ¿Cuántos milisegundos tarda el colector en procesar los 60 elementos $(10 + 60 \times 25)$?
4 [**PRB-0042**] **(Intermedio, Diseño)** Un proyecto Kotlin Multiplatform comparte 14 000 líneas de lógica de dominio en `commonMain` entre Android e iOS, requiriendo solo 1500 líneas específicas en `androidMain` y 1500 en `iosMain`. ¿Cuántas líneas de código duplicado se ahorran respecto a escribir los 14 000 líneas dos veces?

Capítulo 8

Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

*Este capítulo cierra el Volumen VI con la monografía completa (Bloques A – H) de **Swift (6.0+)**. Creado por Chris Lattner y el equipo de Apple sobre la infraestructura LLVM, Swift es un lenguaje compilado nativamente sin recolector de basura de trazado que combina semántica de valor profunda, conteo automático de referencias (**ARC**) y verificación estática de aislamiento de actores en tiempo de compilación.*

Consejo

En firmas de funciones en Swift, prefiere siempre `func render(item: some Drawable)` sobre `func render(item: any Drawable)` salvo que necesites almacenar elementos heterogéneos en un mismo array `[any Drawable]`: `some` permite al compilador especializar la función y evitar el contenedor existencial.

Definición

Aislamiento de Actor (`actor` en Swift): tipo por referencia que protege su estado mutable encapsulado mediante un ejecutor serie exclusivo, exigiendo estáticamente en tiempo de compilación que cualquier acceso desde fuera del actor sea asíncrono (`await`) y que los valores transferidos a través de su frontera cumplan `Sendable`.

Autoficha

¿Cómo ocurre la *reentrancia de actores* (*actor reentrancy*) en Swift cuando un método de un `actor` ejecuta un punto de suspensión `await` intermedio, y por qué deben revalidarse las invariantes del actor después del `await`?

8.1 Bloques A–D: Filosofía de Swift, SIL/LLVM, SwiftPM, semántica de valor (struct/enum), Copy-on-Write y protocolos

A diferencia de Java o Objective-C, en Swift casi todos los tipos fundamentales de la biblioteca estándar –incluyendo enteros, flotantes, `String`, `Array<T>`, `Dictionary<K, V>` y `Set<T>`– son **tipos por valor** (`struct`), no clases (`class`). Para que pasar un `Array<Double>` de un millón de elementos por valor a una función no implique copiar el búfer inmediatamente, las colecciones de Swift implementan **Copy-on-Write (CoW)**: al asignar `var b = a`, ambas estructuras comparten el mismo búfer interno en el montículo; solo cuando se ejecuta una mutación como `b.append(1.0)`, `Array` comprueba en $O(1)$ mediante `isKnownUniquelyReferenced` si el contador de referencias del búfer es mayor que 1 y, únicamente en ese caso, duplica el búfer antes de modificarlo. En el diseño con protocolos (`protocol`), Swift 5.7+/6.0 distingue explícitamente entre `some Protocol` (tipo opaco conocido estáticamente en compilación, despachado sin coste) y `any Protocol` (contenedor existencial dinámico de 5 palabras máquina: 3 palabras de búfer de valor en línea más metadatos de tipo y *witness table*).

8.2 Bloques E–H: Gestión de memoria ARC (weak/unowned), concurrencia estricta en Swift 6 (actor, Sendable) y proyectos

Para las clases (`class`) y clausuras, Swift gestiona la memoria de forma determinista en tiempo de compilación mediante **ARC** (*Automatic Reference Counting*): el compilador inserta llamadas `swift_retain` y `swift_release` en la representación intermedia SIL. Como el conteo de referencias no recolecta ciclos aislados, todo ciclo entre instancias de clases o clausuras que capturan `self` debe romperse declarando una de las aristas como `weak` (opcional `T?` que se pone automáticamente a `nil` al destruirse el objeto) o `unowned` (no opcional, cuando ambas vidas útiles están ligadas). Finalmente, **Swift 6** garantiza en tiempo de compilación la ausencia de carreras de datos mediante el protocolo marcador `Sendable` y el tipo de referencia **`actor`**, que serializa el acceso a su estado mutable interno exigiendo `await` desde cualquier contexto externo:

```
public actor TelemetryVault {
    private var readings: [String: Double] = [:]
    public init() {}
    public func record(sensor: String, value: Double) {
        readings[sensor] = value
    }
    public func current(sensor: String) -> Double? {
        return readings[sensor]
    }
}
```

El capítulo concluye con el ecosistema Swift Package Manager (`Package.swift`), `SwiftTesting` (`@Test`, `#expect`), `SwiftUI` / `Vapor` en servidor Linux y cuatro proyectos completos en Swift 6.

Problema resuelto

Enunciado En Swift sobre una arquitectura de 64 bits ($W = 8$ B), un contenedor existencial `any P` para un protocolo ocupa $5 \times 8 = 40$ B (24 B de búfer de valor en línea, 8 B de *Value Witness Table* y 8 B de *Protocol Witness Table*), mientras que un valor concreto `Int64` pasado mediante un genérico especializado `some P` ocupa únicamente 8 B. Calcula cuántos bytes ahorra un array de $N = 10000$ elementos al usar el tipo especializado `[Int64]` frente a un array existencial `[any P]`.

Datos

- $N = 10000$ — Número de elementos en la colección
- $S_{any} = 40\,\text{B}$ — Tamaño del contenedor existencial de 5 palabras
- $S_{some} = 8\,\text{B}$ — Tamaño del tipo concreto especializado

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Sobrecoste espacial de contenedores existenciales (any P) frente a tipos concretos (some P) en Swift

$$\Delta M = 10000 \cdot (40 - 8)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 10000 elementos por los 32 B ahorrados por elemento

$$\Delta M = 10000 \cdot (40 - 8)$$

$$(10000 \cdot (40 - 8))$$

Resultado

$$Resultado = 320000$$

Interpretación La especialización estática ahorra 320 000 bytes (un 80 % de reducción) y elimina el despacho indirecto por tabla de testigos.
Comprobación de los pasos

- Paso 1: 320000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) En Swift de 64 bits ($W = 8$ B), el `Value Buffer` interno de un contenedor existencial tiene 3 palabras. ¿Hasta cuántos bytes puede medir un `struct` para caber en línea dentro del contenedor existencial sin alojar en el montículo (3×8)?
2 [PRB-0046] (Básico, Numérico) Si se copia un `Array<Int>` de 50 000 enteros (`var b = a`) y solo se realizan lecturas sobre `a` y `b` sin mutar ninguno de los dos, ¿cuántos elementos se duplican en el montículo gracias a Copy-on-Write?
3 [PRB-0047] (Intermedio, Desarrollo por pasos) Un contenedor existencial para una composición de $K = 3$ protocolos (`any P1 & P2 & P3`) ocupa $(3 + 1 + 3) \times 8$ bytes en 64 bits. ¿Cuántos bytes mide en total?
4 [PRB-0048] (Intermedio, Diseño) En la cabecera de un objeto `class` en Swift de 64 bits, se almacenan el puntero `isa`/metadatos (8 B) y la palabra de contadores ARC (8 B). ¿Cuántos bytes suman las cabeceras de 2000 instancias?

Apéndice A

Notación

Símbolo	Significado	Unidades
$(G,\ M,\ P)$	Goroutine, Machine (hilo OS) y Processor (contexto lógico) en el runtime de Go	entidades
$\mathrm{ARC}(o)$	Contador fuerte/débil/unowned gestionado en compilación en Swift	referencias
$\mathrm{Span}\langle T\rangle$	Vista contigua de memoria alojada en pila (ref struct) sin asignación en el montículo en .NET	vista

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

Problema resuelto [PRB-0001]
En una aplicación financiera en C# sobre .NET de 64 bits, se mantiene un búfer de $N = 100000$ cotizaciones donde cada cotización tiene $S_{\text{val}} = 16$ B de datos (`long Timestamp`, `double Price`). Si se modela como `readonly record struct`, el array `Quote[]` ocupa 100000×16 B. Si se modelara como `class`, cada elemento añadiría 8 B de puntero en el array más 16 B de cabecera de objeto en el montículo (24 B extra por instancia). Calcula cuántos kilobytes (dividiendo entre 1000) ahorra el diseño con `struct`.
Respuesta

$$Resultado = 2400$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
En .NET de 64 bits, la cabecera de todo objeto por referencia (`class`) consta de `ObjHeader` (8 B) y `MethodTable*` (8 B). ¿Cuántos bytes suma esta cabecera?
Respuesta 16
Ejercicio 2 [PRB-0004]
Un `readonly record struct Point3D(double X, double Y, double Z)` contiene tres `double` de 8 bytes. ¿Cuántos bytes ocupa cada instancia?
Respuesta 24
Ejercicio 3 [PRB-0005]
Una consulta LINQ encadena `.Where(...)` y `.Take(25)` sobre una secuencia perezosa de 10 000 elementos donde se cumple el predicado en 1 de cada 4 elementos. ¿Cuántos elementos evalúa antes de detenerse (25×4)?
Respuesta 100
Ejercicio 4 [PRB-0006]
El recolector generacional del CLR divide el montículo de objetos pequeños en 3 generaciones (`Gen 0`, `Gen 1`, `Gen 2`). Si `Gen 0` tiene un presupuesto de 4 MiB y `Gen 1` de 2 MiB, ¿cuántos KiB suman las dos generaciones efímeras ($(4 + 2) \times 1024$)?
Respuesta 6144

Cálculos reproducidos

$\frac{(100000 \cdot 24)}{1000} = 2400$

C.2 Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

Problema resuelto [PRB-0007]
Un servicio de alta frecuencia en ASP.NET Core procesa $R = 50000$ llamadas por segundo donde el 90 % (45000 llamadas/s) se resuelven síncronamente desde memoria local y solo el 10 % (5000 llamadas/s) requieren E/S asíncrona real. Si cada objeto `Task<Result>` alojado en el montículo mide $S_{\text{task}} = 72$ B, calcula cuántos kilobytes por segundo (dividiendo entre 1000) deja de alojar en `Gen 0` el servicio al cambiar el tipo de retorno de `Task<Result>` a `ValueTask<Result>`.

Respuesta

$$Resultado = 3240$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
En .NET de 64 bits, un `Span<byte>` contiene una referencia interior `ref byte` (8 B) y una longitud `int` alineada a 8 B. ¿Cuántos bytes ocupa en la pila?
Respuesta 16

Ejercicio 2 [PRB-0010]
Si se alquila un búfer con `ArrayPool<byte>.Shared.Rent(4000)`, el pool devuelve un array cuya longitud es la siguiente potencia de dos (4096). ¿Cuántos bytes adicionales respecto a los 4000 solicitados tiene el array devuelto?
Respuesta 96

Ejercicio 3 [PRB-0011]
Un canal acotado `Channel.CreateBounded<Job>(new BoundedChannelOptions(128))` contiene actualmente 95 trabajos. ¿Cuántos trabajos más pueden escribirse sin suspender al productor?
Respuesta 33

Ejercicio 4 [PRB-0012]
Un contenedor de inyección de dependencias resuelve por cada petición HTTP 1 servicio `Singleton` (ya creado), 2 servicios `Scoped` y 4 servicios `Transient` (invocados 2 veces cada uno en la petición). ¿Cuántas instancias nuevas se crean durante esa petición $(2 + 4 \times 2)$?
Respuesta 10

Cálculos reproducidos

$\frac{(45000 \cdot 72)}{1000} = 3240$

C.3 Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

Problema resuelto [PRB-0013]
En la JVM HotSpot de 64 bits con punteros comprimidos activos (`-XX:+UseCompressedOops`), un objeto `Double` encajonado (*boxed*) ocupa 24 B (12 B de cabecera + 8 B del `double` + 4 B de relleno a múltiplo de 8), más 4 B de la referencia en un `ArrayList<Double>` (28 B por número), mientras que un array primitivo `double[]` o un `DoubleStream` ocupa únicamente 8 B por número. Calcula cuántos megabytes (dividiendo entre 10^6) se ahorran al almacenar $N = 2000000$ de valores en `double[]` frente a `ArrayList<Double>`.
Respuesta

$$Resultado = 40$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0015]
En la JVM HotSpot con Compressed OOPs, la cabecera de un objeto ordinario mide 8 B (`Mark Word`) más 4 B (`Klass Pointer`). ¿Cuántos bytes mide la cabecera antes de alinear campos?
Respuesta 12

Ejercicio 2 [PRB-0016]
En `HashMap` de Java 8+, cuando un cubo (*bucket*) alcanza el umbral `TREEIFY_THRESHOLD = 8` colisiones (con capacidad de tabla ≥ 64), la lista enlazada se transforma en un árbol rojo-negro. Si hay 256 elementos colisionando en ese mismo cubo, ¿cuál es la altura máxima $2 \log_2(256)$ a comparar frente a los 256 saltos lineales?
Respuesta 16

Ejercicio 3 [PRB-0017]

Un `ConcurrentHashMap` redimensiona su tabla de 1024 a 2048 cubos manteniendo el factor de carga en 0,75. ¿Cuántas entradas admite la nueva tabla antes del siguiente redimensionado ($2048 \times 3/4$)?

Respuesta 1536

Ejercicio 4 [PRB-0018]

Si el tamaño de un montículo de la JVM supera los 32 GiB y desactiva Compressed OOPs, cada referencia pasa de 4 B a 8 B. En un grafo en memoria con 50 millones de referencias, ¿cuántos megabytes (10^6 B) adicionales consumen las referencias?

Respuesta 200

Cálculos reproducidos

$\frac{(2000000 \cdot (28-8))}{1000000} = 40$

C.4 Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

Problema resuelto [PRB-0019]

Según la Ley de Little ($\lambda_{\text{máx}} = L_{\text{máx}}/W$), un microservicio donde cada petición pasa $W = 0{,}25$ s esperando respuestas de APIs externas está limitado con hilos tradicionales de plataforma a $L_{\text{plat}} = 400$ hilos (caudal máximo $400/0{,}25 = 1600$ req/s). Al migrar a **Virtual Threads** en Java 21 soportando $L_{\text{virt}} = 20000$ hilos virtuales simultáneos con la misma latencia $W = 0{,}25$ s, calcula cuántas peticiones por segundo adicionales puede sostener el servidor ($20000 / 0.25 - 400 / 0.25$).

Respuesta

$$\textit{Resultado} = 78400$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0021]

Por la Ley de Little ($L = \lambda \cdot W$), si un sistema recibe $\lambda = 2500$ peticiones/s y cada una tarda $W = 0{,}2$ segundos, ¿cuántas peticiones hay en vuelo simultáneamente en promedio?

Respuesta 500

Ejercicio 2 [PRB-0022]

En una CPU de 16 núcleos lógicos, el `ForkJoinPool` portador de hilos virtuales crea por defecto un hilo portador (*carrier thread*) por núcleo. ¿Cuántos hilos del sistema operativo mantienen activos los hilos virtuales?

Respuesta 16

Ejercicio 3 [PRB-0023]

En el código `VirtualThreadDispatcher.runConcurrentTasks(100)`, se suma `i * 2` para $i = 0, 1, \ldots, 99$. ¿Cuál es el valor exacto de la suma (99×100)?

Respuesta 9900

Ejercicio 4 [PRB-0024]

Una `Arena.ofConfined()` de Project Panama reserva 64 segmentos nativos de 4096 bytes cada uno. ¿Cuántos bytes de memoria *off-heap* se liberan determinísticamente al cerrarse el bloque `try-with-resources`?

Respuesta 262144

Cálculos reproducidos

$\left(\frac{20000}{0{,}25} - \frac{400}{0{,}25}\right) = 78400$

C.5 Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

Problema resuelto [PRB-0025]
En un programa en Go, se crea un slice `base := make([]int64, 100, 200)` (cada `int64` ocupa 8 B) y se obtiene la sub-rebanada de tres índices `sub := base[20:60:80]`. Calcula la suma de la longitud `len(sub)` ($60 - 20$) más la capacidad `cap(sub)` ($80 - 20$) multiplicada por los 8 B de cada elemento.
Respuesta

$$Resultado = 800$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En Go de 64 bits, el encabezado de un slice (`SliceHeader`) consta de `Data` (8 B), `Len` (8 B) y `Cap` (8 B). ¿Cuántos bytes ocupa en la pila?
Respuesta 24
Ejercicio 2 [PRB-0028]
En Go, cada cubo (*bucket* `bmap`) de un `map[K]V` almacena exactamente 8 pares clave-valor. Si un mapa tiene $2^6 = 64$ cubos, ¿cuántos pares clave-valor caben en esos 64 cubos sin cubos de desbordamiento (64×8)?
Respuesta 512
Ejercicio 3 [PRB-0029]
Para la rebanada `s[15:45:50]`, ¿cuántos elementos pueden añadirse con `append` antes de superar la capacidad y forzar la reserva de un nuevo array $((50 - 15) - (45 - 15))$?
Respuesta 5
Ejercicio 4 [PRB-0030]
Una interfaz `io.Reader` en Go se representa como un par de palabras `(itab*, data*)` de 8 bytes cada una. ¿Cuántos bytes ocupan 500 descriptores de interfaz en un slice `[]io.Reader`?
Respuesta 8000

Cálculos reproducidos

$(((60 - 20) + (80 - 20)) \cdot 8) = 800$

C.6 Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

Problema resuelto [PRB-0031]
Cada goroutine recién creada en Go 1.22 reserva una pila inicial de $S_G = 2$ KiB (2048 B), mientras que un hilo nativo de sistema operativo reserva por defecto una pila de $S_M = 2048$ KiB (2 MiB). Si un servidor de telemetría en Go mantiene $N = 100000$ goroutines activas sobre $M = 16$ hilos del sistema operativo, calcula cuántos mebibytes totales (dividiendo los KiB entre 1024) ocupan las pilas de las 100 000 goroutines más los 16 hilos OS.
Respuesta

$$Resultado = 227{,}3125$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
¿Cuántos bytes mide la pila inicial de una goroutine en Go (2×1024)?
Respuesta 2048
Ejercicio 2 [PRB-0034]
En el planificador de Go, un procesador P_1 tiene 180 goroutines en su cola local y un procesador inactivo P_2 le roba la mitad. ¿Cuántas goroutines roba P_2?

Respuesta 90

Ejercicio 3 [PRB-0035]

Un canal con búfer `ch := make(chan uint64, 512)` almacena elementos de 8 bytes. Si hay 300 elementos encolados en el búfer, ¿cuántos elementos adicionales caben antes de bloquear al emisor?

Respuesta 212

Ejercicio 4 [PRB-0036]

En el problema anterior, ¿cuántos bytes ocupa el búfer circular completo de 512 elementos `uint64` (512×8)?

Respuesta 4096

Cálculos reproducidos

$\frac{((100000 \cdot 2)+(16 \cdot 2048))}{1024} = 227{,}3125$

C.7 Monografía de Kotlin (Bloques A−H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

Problema resuelto [PRB-0037]

En una aplicación en Kotlin sobre la JVM, una función de orden superior que captura una variable local del entorno se invoca en un bucle caliente $N = 500000$ veces por segundo. Si la función NO está marcada como `inline`, cada invocación aloja una instancia de clase anónima de $S_{\text{lambda}} = 24$ B. Al marcar la función de orden superior con `inline`, el alojamiento cae a 0 B. Calcula cuántos megabytes por segundo (dividiendo entre $\$10^6\$$) deja de alojar el bucle gracias a `inline`.

Respuesta

$$Resultado = 12$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0039]

En la función `aggregateClusterLatency` con `nodeIds = [1, 2, 3, 4, 5]` donde cada nodo devuelve `id * 5`, ¿cuál es la suma total devuelta ($(1+2+3+4+5) \times 5$)?

Respuesta 75

Ejercicio 2 [PRB-0040]

Un `@JvmInline value class Meter(val v: Double)` se almacena en un array primitivo de 1000 elementos. ¿Cuántos bytes ocupan los 1000 elementos `double` (1000×8)?

Respuesta 8000

Ejercicio 3 [PRB-0041]

Un `Flow<Int>` emite 60 elementos a razón de 1 elemento cada 10 ms y el colector aplica `.buffer(10)` tardando 25 ms por elemento. ¿Cuántos milisegundos tarda el colector en procesar los 60 elementos ($10 + 60 \times 25$)?

Respuesta 1510

Ejercicio 4 [PRB-0042]

Un proyecto Kotlin Multiplatform comparte 14 000 líneas de lógica de dominio en `commonMain` entre Android e iOS, requiriendo solo 1500 líneas específicas en `androidMain` y 1500 en `iosMain`. ¿Cuántas líneas de código duplicado se ahorran respecto a escribir los 14 000 líneas dos veces?

Respuesta 14000

Cálculos reproducidos

$\frac{(500000 \cdot 24)}{1000000} = 12$

C.8 Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

Problema resuelto [PRB-0043]
En Swift sobre una arquitectura de 64 bits ($W = 8$ B), un contenedor existencial `any P` para un protocolo ocupa $5 \times 8 = 40$ B (24 B de búfer de valor en línea, 8 B de *Value Witness Table* y 8 B de *Protocol Witness Table*), mientras que un valor concreto `Int64` pasado mediante un genérico especializado `some P` ocupa únicamente 8 B. Calcula cuántos bytes ahorra un array de $N = 10000$ elementos al usar el tipo especializado `[Int64]` frente a un array existencial `[any P]`.
Respuesta

$$Resultado = 320000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
En Swift de 64 bits ($W = 8$ B), el `Value Buffer` interno de un contenedor existencial tiene 3 palabras. ¿Hasta cuántos bytes puede medir un `struct` para caber en línea dentro del contenedor existencial sin alojar en el montículo (3×8)?
Respuesta 24
Ejercicio 2 [PRB-0046]
Si se copia un `Array<Int>` de 50 000 enteros (`var b = a`) y solo se realizan lecturas sobre `a` y `b` sin mutar ninguno de los dos, ¿cuántos elementos se duplican en el montículo gracias a Copy-on-Write?
Respuesta 0
Ejercicio 3 [PRB-0047]
Un contenedor existencial para una composición de $K = 3$ protocolos (`any P1 & P2 & P3`) ocupa $(3 + 1 + 3) \times 8$ bytes en 64 bits. ¿Cuántos bytes mide en total?
Respuesta 56
Ejercicio 4 [PRB-0048]
En la cabecera de un objeto `class` en Swift de 64 bits, se almacenan el puntero `isa`/metadatos (8 B) y la palabra de contadores ARC (8 B). ¿Cuántos bytes suman las cabeceras de 2000 instancias?
Respuesta 32000

Cálculos reproducidos

$(10000 \cdot (40 - 8)) = 320000$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 31 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 1 respaldada por una fuente.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-csharp-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-gamma-gof (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-java-jls21 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Go Programming Language Specification & Memory Model (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶25, ¶26, ¶27, ¶28, ¶29, ¶30, ¶31, ¶32, ¶33, ¶34, ¶35, ¶36.
Referencia Normativa src-kotlin-spec (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
The Swift Programming Language (Swift 6.0 Official Book) (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
REV-0001: cap01-csharp-dotnet-clr-tipos-valor-referencia-linq scientific
REV-0002: cap02-csharp-avanzado-experto-span-async-valuetask-aspnet-proyectos scientific
REV-0003: cap03-java-jvm-bytecode-memoria-generics-streams scientific
REV-0004: cap04-java-avanzado-experto-virtual-threads-jmm-spring-proyectos scientific
REV-0005: cap05-go-fundamentos-slices-interfaces-errores-genericos scientific
REV-0006: cap06-go-avanzado-experto-goroutines-gmp-canales-context-proyectos scientific
REV-0007: cap07-monografia-kotlin-null-safety-coroutines-flow-kmp scientific
REV-0008: cap08-monografia-swift-valor-cow-protocolos-arc-actores scientific

Apéndice E

Plan de estudio

Monografías de Plataformas Gestionadas: C# (.NET 8/9) y Java (JVM 21 LTS)

✓Monografía de C# (I – Bloques A–D): Arquitectura .NET/CLR, Tipos Valor vs Referencia, Records, Pattern Matching y LINQ

- Comprender la arquitectura del Common Language Runtime (CLR), el lenguaje intermedio CIL, la compilación JIT (RyuJIT) y Native AOT en .NET 8/9.
- Distinguir con precisión semántica y espacial entre tipos por valor (‘struct‘, ‘readonly struct‘, ‘ref struct‘) y tipos por referencia (‘class‘, ‘record‘), evitando el boxing involuntario.
- Dominar la nulabilidad verificada (‘string¿), coincidencia de patrones avanzada y consultas declarativas con LINQ (‘IEnumerable<T>‘ vs ‘IQueryable<T>‘).

2 conceptos planificados.

✓Monografía de C# (II – Bloques E–H): Span<T>, Memory<T>, Async/Await, ValueTask, ASP.NET Core y Proyectos

- Programar procesamiento de alto rendimiento sin asignaciones en el montículo usando ‘Span<T>‘, ‘ReadOnlySpan<T>‘, ‘Memory<T>‘, ‘ArrayPool<T>‘ y ‘System.IO.Pipelines‘.
- Comprender la máquina de estados generada por el compilador para ‘async‘/‘await‘, cuándo utilizar ‘ValueTask<T>‘ frente a ‘Task<T>‘ y la propagación de ‘CancellationToken‘.
- Dominar el ecosistema empresarial de .NET (ASP.NET Core, inyección de dependencias, EF Core, Dapper, xUnit, BenchmarkDotNet) y los 4 proyectos integradores.

2 conceptos planificados.

✓Monografía de Java (I – Bloques A–D): Arquitectura JVM, Bytecode, Memoria, Records, Sealed Classes, Generics y Streams

- Explicar la arquitectura de la Java Virtual Machine (JVM): subsistema ClassLoader, áreas de memoria (Heap, Metaspace, pilas de hilo), intérprete y compiladores JIT C1/C2.
- Dominar las innovaciones de Java 17/21 LTS: ‘record‘, ‘sealed interface‘/‘permits‘, coincidencia de patrones en ‘switch‘ y desestructuración de records.
- Aplicar genéricos con comodines acotados (principio PECS: Producer Extends, Consumer Super), colecciones (‘ArrayList‘, ‘HashMap‘, ‘ConcurrentHashMap‘) y la API ‘Stream<T>‘.

2 conceptos planificados.

✓Monografía de Java (II – Bloques E–H): Java Memory Model, Virtual Threads (Loom), Foreign Function & Memory API y Proyectos

- Dominar el Java Memory Model (JMM, JLS §17.4): la relación de orden parcial *happens-before*, ‘volatile‘, ‘VarHandle‘, ‘CompletableFuture‘ y ‘ReentrantLock‘.
- Escalar aplicaciones síncronas legibles a millones de tareas concurrentes mediante **Virtual Threads** (JEP 444 en Java 21 LTS) y Concurrencia Estructurada.

- Integrar el ecosistema empresarial de Java (Maven, Gradle, JUnit 5, Testcontainers, Spring Boot 3, Micrometer/OpenTelemetry) y construir los 4 proyectos del bloque.

2 conceptos planificados.

Monografías de Sistemas Cloud y Móviles: Go, Kotlin y Swift

✓Monografía de Go (I – Bloques A–D): Filosofía, Compilación Estática, Slices, Maps, Interfaces Implícitas y Manejo de Errores

- Comprender la filosofía de simplicidad ortogonal de Go (Rob Pike, Ken Thompson, Robert Griesemer), su compilador ultrarrápido y el enlazado de binarios estáticos.
- Dominar la anatomía en memoria de ‘slice‘ (puntero, longitud, capacidad) y ‘map‘, el valor cero útil (*zero value*), composición de ‘struct‘ por incrustación (*embedding*) y ‘defer‘.
- Diseñar abstracciones desacopladas mediante interfaces satisfechas implícitamente, manejo explícito de errores como valores (‘errors.Is‘, ‘errors.As‘, ‘ %w‘) y genéricos de Go 1.18+.

2 conceptos planificados.

✓Monografía de Go (II – Bloques E–H): Planificador G-M-P, Goroutines, Canales, Context, Perfilado pprof y Proyectos

- Explicar el funcionamiento interno del planificador M:N con robo de trabajo (*work-stealing*) de Go basado en las entidades G (Goroutine), M (Machine) y P (Processor).
- Diseñar patrones concurrentes basados en CSP (Tony Hoare) con canales con y sin búfer, ‘select‘, cancelación jerárquica con ‘context.Context‘ y ‘errgroup.Group‘.
- Auditar el escape al montículo (‘go build -gcflags='-m'‘), detectar carreras de datos (‘go test -race‘), perfilar CPU/memoria con ‘pprof‘ y construir los 4 proyectos en Go.

2 conceptos planificados.

✓Monografía de Kotlin (Bloques A–H): Null-Safety, Data/Sealed Classes, Extensiones, Coroutines, Flow y Multiplatform

- Dominar el sistema de tipos de Kotlin 2.0 (compilador K2): distinción estricta ‘T‘ vs ‘T¿, smart casts, ‘data class‘, ‘sealed interface‘, ‘value class‘ y delegación (‘by‘).
- Escribir código expresivo con funciones de orden superior ‘inline‘, tipos genéricos reificados (‘reified T‘), funciones de extensión y DSLs con receptores (‘T.() -> Unit‘).
- Construir sistemas concurrentes y reactivos con **Kotlin Coroutines**: funciones ‘suspend‘, transformaciones CPS, ‘CoroutineScope‘, ‘Dispatchers‘, ‘Structured Concurrency‘ y ‘Flow‘ / ‘StateFlow‘.

2 conceptos planificados.

✓Monografía de Swift 6 (Bloques A–H): Semántica de Valor, Copy-on-Write, Protocolos, ARC y Concurrencia con Actores

- Comprender la arquitectura del compilador ‘swiftc‘ (SIL: Swift Intermediate Language y LLVM), la semántica de valor (‘struct‘, ‘enum‘) y la optimización Copy-on-Write (CoW).
- Diseñar abstracciones con Programación Orientada a Protocolos (‘protocol‘, tipos asociados, tipos opacos ‘some P‘ vs existenciales ‘any P‘).
- Dominar la gestión determinista de memoria mediante **ARC** (‘strong‘, ‘weak‘, ‘unowned‘) y el aislamiento estricto de carreras de datos de **Swift 6** (‘async‘/‘await‘, ‘actor‘, ‘@MainActor‘, ‘Sendable‘).

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-csharp-spec. https://doc.rust-lang.org/reference/src-csharp-spec.html

[27] Referencia Normativa src-gamma-gof. https://doc.rust-lang.org/reference/src-gamma-gof.html

[28] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[29] Referencia Normativa src-java-jls21. https://doc.rust-lang.org/reference/src-java-jls21.html

[30] Referencia Normativa src-kotlin-spec. https://doc.rust-lang.org/reference/src-kotlin-spec.html

[31] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html